2.1 Именные функции

Функции - это самостоятельные блоки кода, которые позволяют структурировать программу, избегать повторения одних и тех же инструкций и делать код более читаемым и удобным для поддержки. Благодаря функциям можно разбивать сложные задачи на небольшие части, каждая из которых решает свою подзадачу.

В Python для создания функции используется ключевое слово def, за которым следует имя функции и круглые скобки с перечнем параметров (если они нужны). Тело функции записывается с отступом.Пример:

```
def greet():
    print("Привет, мир!")
```

В примере представлена простейшая функция, которая принимает два параметра. В зависимости от типов данных параметров, результатом функции может быть число или строка. Инструкция **return** указывает, что функция должна вернуть.

Функция может содержать вложенные функции и возвращать объекты различных типов (списки, словари, функции).

Структура функции определяется спецификой задачи. Оператор **return** не используется, если функция выполняет некоторые действия, но не возвращает значения. В этом случае возвращаемое значение — **None**.

Пример:

```
def my_func():
    pass

print(my_func())
```

Результат:

```
None
```

Здесь в теле функции реализован оператор-заглушка. Его использование равноценно отсутствию операции. **Pass** может применяться в тех случаях, когда код на текущий момент не написан.

Оператор return

О назначении этого оператора уже говорилось выше. Функции могут принимать данные и возвращать определённый результат после их обработки. При этом для выхода из функции и передачи результата в точку вызова применяется оператор **return**.

return со значением

Если при выполнении логики функции интерпретатор Python встречает оператор **return**, то забирает значение, определённое после оператора, и выполняет выход из функции.

Рассмотрим следующий пример — расчёт периметра круга:

Пример:

```
def square_circle(radius):
  square = 2 * 3.14 * radius
  return square

user_radius = float(input('Enter radius '))
result = square_circle(user_radius)
print(round(result,2))
```

Результат:

```
Enter radius 3
18.84
```

Важно! *В Python любая функция что-то возвращает. Если оператор* ***return*** *отсутствует, то возвращаемое значение —* ***None.***

Возврат набора значений

В Python возможно использование оператора **return**, возвращающего из функции несколько объектов. Достаточно указать их через запятую после оператора **return**.

Пример:

```
def get_statistics(numbers):
    """Возвращает минимум, максимум и среднее значение из списка
чисел."""
    minimum = min(numbers)
    maximum = max(numbers)
    average = sum(numbers) / len(numbers)
    return minimum, maximum, average

# Использование функции
data = [10, 23, 7, 15, 42]
min_val, max_val, avg_val = get_statistics(data)

print("Минимум:", min_val)
print("Максимум:", max_val)
print("Среднее:", avg_val)
```

Результат:

```
Минимум: 7
Максимум: 42
Среднее: 19.4
```

Функция get_statistics принимает список чисел. Она вычисляет минимум, максимум и среднее значение. Возвращает эти три значения одновременно.

При вызове функции результат удобно распаковать в три переменные.Смысл в том, что перечисление значений через запятую формирует объект типа кортеж (tuple). Присваивая кортеж сразу набору

переменных, его элементы сопоставляются переменным. Происходит своего рода распаковка.

То есть, когда функция возвращает набор объектов, на деле она возвращает объект-кортеж с этими объектами. Они упаковываются в кортеж перед возвратом. Если за оператором **return** следует только одна переменная, её тип сохраняется в исходном состоянии.

Аргументы функций

Функция может принимать любое количество параметров или не принимать их вообще. Параметры могут быть позиционные и именованные, обязательные и необязательные.

Пример:

```
# позиционные параметры
def first_func(var_1, var_2, var_3):
    return var_1 + var_2 + var_3

print(first_func(10, 20, 30))

# именованные параметры
def second_func(var_2, var_1, var_3):
    print(f"var_2 - {var_2}; var_1 - {var_1}; var_3 - {var_3}")

second_func(var_1=10, var_2=20, var_3=30)
```

Результат:

```
60
var_2 - 20; var_1 - 10; var_3 - 30
```

Пример:

```
    # обязательные параметры
def first_func(var_1, var_2, var_3):
    return var_1 + var_2 + var_3

print(first_func(10, 20, 30))

# var_2 и var_3 - необязательные параметры
def second_func(var_1, var_2=20, var_3=30):
    return var_1 + var_2 + var_3

print(second_func(10))
```

Результат:

```
60
60
```

Функция может принимать неопределённое число позиционных параметров. В этом случае при описании функции используется конструкция ***args**.

Пример:

```
def args_fn(*numbers):
  for arg in numbers:
    print(arg)

args_fn('3','4', '5', '9')
```

Результат:

```
3
4
5
9
```

Из примера следует, что **args** представляет собой кортеж, содержащий переданные в функцию аргументы. С переменной **args** можно выполнять те же операции, что и с кортежем.

Функция может принимать и неопределённое число именованных параметров. Тогда используется конструкция ****kwargs**.

Пример:

```
def kwargs_fn(**numbers):
  for k, v in numbers.items():
    print(k,":", v)

kwargs_fn(one = 100, two = 200, three = 500)
```

Результат:

```
one : 100
two : 200
three : 500
```

Переменная kwargs хранит словарь. С ним можно выполнять привычные для словаря операции.

Важно! Операторы ***** и ****** в Python можно использовать и с другими именами переменных. То есть имена **args** и **kwargs** необязательны. Но помните, что хороший стиль программирования подразумевает использование имён **args** и **kwargs**, так как сразу становится понятно о назначении таких переменных.

2.2 Анонимные функции (lambda)

Это функции, содержащие только одно выражение, но выполняющиеся быстрее именных функций. При этом используется оператор **lambda**. При использовании **lambda**-функций их необязательно присваивать некоторой

переменной, как в случае с именными функциями. **lambda**-функции, в отличие от именных, не требуют оператора **return**, в остальном — идентичны именным.

Пример:

```
square = lambda x: x * x
print(square(4))  # 16
```

Функция range() для многократно выполняемых действий. Отвечает за генерацию набора чисел в пределах указанного диапазона. Для этого можно использовать ещё один параметр — шаг. Пример:

```
print(list(range(7))) # целые числа в диапазоне [0, 7)
print(list(range(2, 8))) # целые числа в диапазоне [2, 8)
print(list(range(1, 9, 2))) # целые числа в диапазоне [1, 9) с
шагом 2
print(list(range(1, -7, -2))) # целые числа в диапазоне [1, -7) с
шагом -2
print(list(range(0))) # целые числа в диапазоне (0, 0)
print(list(range(1, 0))) # целые числа в диапазоне (1, 0)
```

Результат:

```
[0, 1, 2, 3, 4, 5, 6]
[2, 3, 4, 5, 6, 7]
[1, 3, 5, 7]
[1, -1, -3, -5]
[]
[]
```

Области видимости переменных в функциях

Понятие «Область видимости» определяет, когда и в каких точках программы разработчик может применять свои пользовательские объекты (переменные, функции). В Python доступны следующие области видимости: локальная, глобальная, нелокальная.

Локальная область видимости. Переменная, объявленная в рамках функции, по умолчанию имеет локальную область видимости. Рассмотрим ещё раз пример, представленный ранее. Пример:

```
x = 10
def fn(x):
  print(x)#0
  x = 20
  print(x)#20

print(x)#10
fn(0)
print(x)#10
```

Результат:

```
10
0
20
10
```

В этом примере объявляем переменную x и присваиваем ей значение 10, вызываем функцию и передаем значение аргумента 0. Внутри функции происходит вывод значения переменной x, равной 0, т.к. это переменная уже из локальной области и не имеет ничего общего с переменной x глобальной области, равной 10.

Глобальная область видимости. Оператор **global** позволяет определить глобальную область видимости для переменной, объявленной в рамках функции. *Использование глобальной переменной должно быть очень аргументированным.* Пример:

```
x = 10
def fn():
  global x
  print(x)
  x = 20

fn()
```

Результат:

```
10
```

2.3 Полезные инструменты

Импорт модуля из стандартной библиотеки

Для этого применяется оператор **import**, за которым следует название модуля. С помощью одной инструкции импорта можно подключить к программе сразу несколько модулей. Но это ухудшает читаемость кода и не соответствует соглашениям PEP-8. Поэтому импортировать следует каждый модуль отдельно.

Рассмотрим применение оператора random с применением модулей **random** и **time**. Пример:

```
import time
import random
print(time.time())
print(random.random())
```

Результат:

```
1563440619.2266152
0.7303585873639512
```

После импорта модуля его имя можно использовать как переменную,

через которую доступны параметры и функции модуля. В примере, рассмотренном выше, импортируются модули целиком. Можно импортировать только определённые объекты модуля. Пример:

```
from time import time
from random import random
print(time())
print(random())
```

Результат:

```
1563441483.3917782
0.5331559021496495
```

Модуль random как генератор псевдослучайных чисел

Модуль содержит специальные функции для генерации целых и дробных чисел. Рассмотрим использование этих функций на примерах.

Генерация целых случайных чисел. Применяются функции **randint()** и **randrange()**. Первая — самая простая в использовании. Она принимает два аргумента — нижняя и верхняя границы целочисленного диапазона, из которого выбирается число. Пример:

```
import random
print(random.randint(0, 10))
```

Результат:

```
7
```

Для функции **randint()** значения и нижней, и верхней границы входят в диапазон, из которого определяется число. Можно работать с функцией напрямую, импортируя из модуля. Пример:

```
from random import randint
print(randint(0, 10))
```

Результат:

```
10
```

Левая граница всегда должна быть меньше правой. Допускается использование отрицательных чисел для определения границ диапазона. Пример:

```
from random import randint
print(randint(-100, -10))
```

Результат:

```
-78
```

Функция **randrange()** устроена сложнее. Она может принимать от одного до трёх аргументов. Один аргумент — возвращается случайное число от 0 до переданного аргумента. При этом сам аргумент в диапазон не включается. Два аргумента — возвращается случайное число в указанном диапазоне. При этом верхняя граница в диапазон не включается. Три аргумента. Первые два — нижняя и верхняя границы, третий — шаг. Например, для функции randrange(20, 30, 3) случайное число выбирается из чисел 20, 23, 26, 29. Пример:

```
from random import randrange
print(randrange(10)) #5
print(randrange(10, 20)) #17
print(randrange(20, 30, 3)) #26
```

Генерация дробных случайных чисел

Самый простой способ получить вещественное число — применить функцию **random()** без параметров. Результат её работы — число с плавающей точкой от 0 до 1, не включая верхнюю границу диапазона. Пример:

```
from random import random
print(random()) #0.7745718967220968
```

Конструкция yield. Использование конструкции **yield** тесно связано с понятием генератора. Это итерируемый объект, который можно использовать один раз, так как при использовании генератора значения не хранятся в памяти. Они формируются в процессе обращения к ним, по мере запроса. Пример:

```
generator = (param * param for param in range(5))

for el in generator:
    print(el)
```

Результат:

```
0
1
4
9
16
```

Важно, что пройтись по генератору можно только один раз, данные в памяти не хранятся. При повторной попытке возникнет ошибка. Например, можно попытаться получить следующее значение с помощью функции **next()**. Пример:

```
generator = (param * param for param in range(5))

```

```
for el in generator:
    print(el)

print(next(generator))
```

Результат:

```
StopIteration
```

Оператор **yield** по назначению похож с оператором **return**, но возвращает генератор вместо значения. Пример:

```
def generator():
    for el in (10, 20, 30):
        yield el

g = generator()
print(g)

for el in g:
    print(el)
```

Результат:

```
<generator object generator at 0x000000C64E181138>
10
20
30
```

Такой механизм может быть полезен в том случае, когда функция возвращает большой объём данных. Но использовать их нужно только единожды. При вызове функции с оператором **yield** функция не выполняется. Она возвращает объект-генератор, с которым далее можно выполнять нужные действия.

Модуль math. Предоставляет многочисленные функции для работы с числами:

Функции	Назначение
`ceil (N)`	Округлить число N до ближайшего большего числа
`fabs (N)`	Определить модуль числа N
`factorial (N)`	Найти факториал числа N
`floor (N)`	Округлить число вниз
`fmod (a, b)`	Получить остаток от деления a на b

`isfinite (N)`	Является ли N числом
`modf (N)`	Определить дробную и целую часть числа N
`sqrt (N)`	Определить квадратный корень числа N
`sin (N)`	Определить синус для N-радианов
`cos (N)`	Определить косинус для N-радианов
`tan (N)`	Определить тангенс для N-радианов
`degrees (N)`	Перевести радианы в градусы
`radians (N)`	Перевести градусы в радианы

В таблице приведена только часть функций. С полным списком можно ознакомиться по ссылке. Пример:

```
from math import ceil, fabs, factorial, floor, \
   fmod, isfinite, modf, sqrt, sin, cos, tan, degrees, radians

print(f"ceil() -> {ceil(6.75)}")
print(f"fabs() -> {fabs(-4)}")
print(f"factorial() -> {factorial(5)}")
print(f"floor() -> {floor(4.34)}")
print(f"fmod() -> {fmod(9, 4)}")
print(f"isfinite() -> {isfinite(10)}")
print(f"modf() -> {modf(10.5)}")
print(f"sqrt() -> {sqrt(16)}")
print(f"sin() -> {sin(1.5708)}")
print(f"cos() -> {cos(1.5708)}")
print(f"tan() -> {tan(1.5708)}")
print(f"degrees() -> {degrees(1.5708)}")
print(f"radians() -> {radians(90)}")
```

Результат:

```
ceil() -> 7
fabs() -> 4.0
factorial() -> 120
floor() -> 4
fmod() -> 1.0
isfinite() -> True
modf() -> (0.5, 10.0)
sqrt() -> 4.0
sin() -> 0.9999999999932537
cos() -> -3.673205103346574e-06
tan() -> -272241.80840927624
degrees() -> 90.00021045914971
radians() -> 1.5707963267948966
```

Исключения в python. Исключения (exceptions) - ещё один тип данных в python. Исключения необходимы для того, чтобы сообщать программисту об ошибках. Конструкция try..except имеет следующее формальное определение:

Конструкция try..except имеет следующее формальное определение:

```
try:
    инструкции
except [Тип_исключения]:
    инструкции
```

Рассмотрим пример деления на 0:

```
try:
   a = 100
   b = 0
   c = a / b
except ZeroDivisionError as e:
   print(e)
```

Результат

```
division by zero
```

Данная инструкция не полная. Полная форма представлена ниже.

```
try:
    исполняется какой-то код
except Exception as e:
    обработка исключения
else:
    код, который будет исполнен в случае, когда не возникает
    исключения
finally:
    код, который гарантированно будет исполнен последним (всегда
    исполняется)
```

Весь основной код, в котором потенциально может возникнуть исключение, помещается после ключевого слова try. Если в этом коде генерируется исключение, то работа кода в блоке try прерывается, и выполнение переходит в блок except.

После ключевого слова except опционально можно указать, какое исключение будет обрабатываться (например, ValueError или KeyError). После слова except на следующей стоке идут инструкции блока except, выполняемые при возникновении исключения.

Как правило, блок finally применяется для освобождения используемых ресурсов, например, для закрытия файлов.

Список основных исключений и их описания приведены в таблице ниже:

Исключение	Описание
Exception	Любое исключение, не являющееся системным
ZeroDivisionError	Попытка деления на ноль
IndexError	Индекс не входит в диапазон элементов
KeyError	Несуществующий ключ
FileExistsError	Попытка создания существующего файла или директории
FileNotFoundError	Файл или директория не существует
IndentationError	Неправильные отступы
TypeError	Несоответствие объекта и типа данных
ValueError	Некорректное значение аргумента функции